Team Member 1: Bolin Li

Team Member 2: Fadi Mardelli

Complex Digital Design

25 April 2024

Improved multi precision adder

Objectives

The main goal of the final project is to upgrade the design of the multi precision adder introduced in Lab #3. Where the multi-precision adder will use the parametrizable N-bit Ripple-Carry Adder as designed in Lab #1. This involves creating a new combinational adder to replace the current N-bit Ripple-Carry Adder in the multi-precision adder (mp_adder) module. The aim is to improve performance while keeping latency to a minimum. To achieve this, the adder width needs to be increased to 32 bits or more, ensuring that the finite state machine (FSM) can work efficiently and process input operands A and B, each 512 bits long, in fewer clock cycles.

Design summary.

The final adder design combines aspects of both Carry Select Adder (CSelA) and Carry Lookahead Adder (CLA) to create a 128-bit (N) hybrid adder (HYBRID). This design approach was inspired by comparisons of delay calculations discussed in the Complex Digital Design (CDD) lecture.

This hybrid adder can perform addition, subtraction operations on two 512-bit operands without violating any given timing constraints. The number of cycles required to complete these operations remains constant at these two operations, determined by dividing the 512-bit operand width by the 128-bit adder width. Next to these cycles we have two extra cycles. One when the input iStart is asserted, the FSM will copy the operand inputs internally. Another cycle at the end is when the FSM will indicate that the operation is completed and that the result is valid.

Number of cycles (addition) = $1 + \frac{512}{128} + 1 = 6$ cycles.

A subtraction function was added using the adder, by taking the second operands 2's complement number to get a 2's complement result.

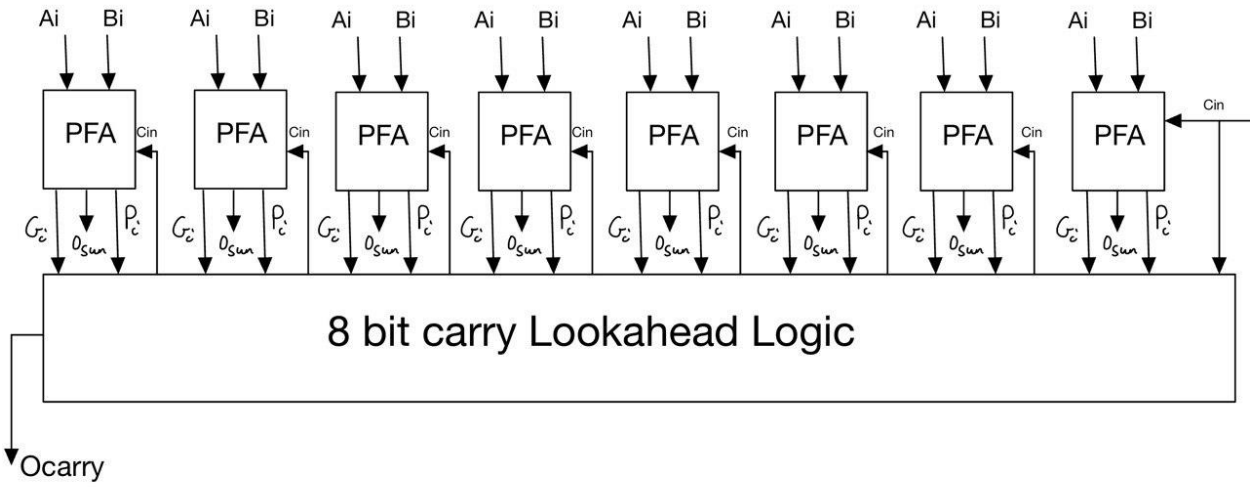
In terms of performance metrics, the worst-negative-delay of the N-bit HYBRID adder is 1.004 ns for addition only 0.687 ns for adder/subtractor operations, these figures are comparable to those of a

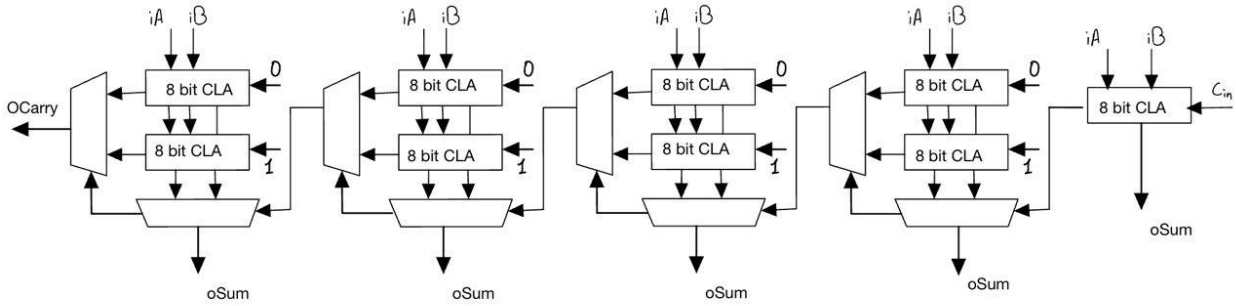
conventional ripple carry adder of the same width. However, it's important to note that further enhancements are possible by incorporating modern architectures such as Spanning Tree adder designs.

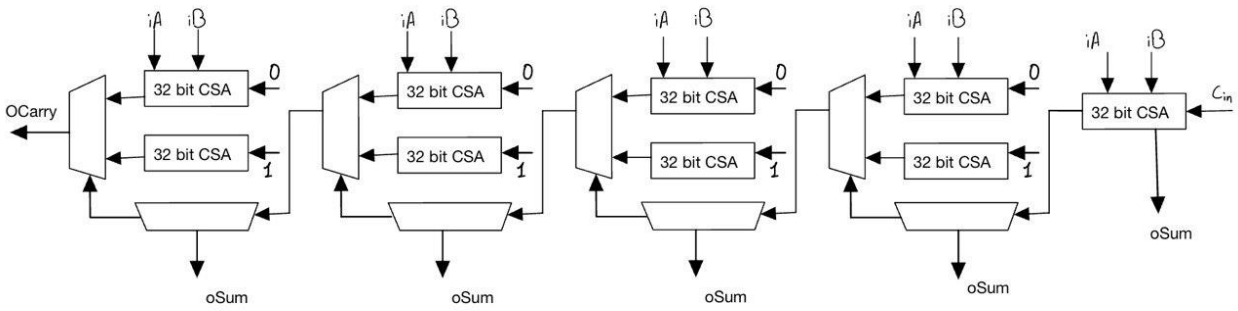
Technical description

1. 128bit adder

In the design we are using a 3-layer implementation method. The bottom layer consists of an 8-bit CLA. On top of that, there are two additional layers of CSA (a balanced design offering good area and speed). However, a large 32-bit CLA composed of iterative 4-bit CLAs experiences poor timing due to prolonged propagation of P and G signals.








```
A     =  0xb311bdd0002247cc8fa7d3b3d3598d0ad16670b771f64b0a7080d162f8b0e96cfe70c101795cc7e2eb8795893e35da2e2d53dbac242e8efcf7ae
15e97d5b4702
B     =  0xd2597675b34b93d99e4d3bbca3e784b6dc82ba91d57baa6aac02960f7d164c6cb8f67503189d82f1221d8c83a0ba80178b30a37395b84af34853
7f2db1cfbeb9
A + B =  0x1856b3445b36ddba62df50f70774111c1ade92b494771f5751c83677275c735d9b767360491fa4ad40da5220cdef05a45b8847f1fb9e6d9f0400
195172f2b05bb
```

```
Result received correctly!
Expected   =  0x1856b3445b36ddba62df50f70774111c1ade92b494771f5751c83677275c735d9b767360491fa4ad40da5220cdef05a45b8847f1fb9e6d9
f0400195172f2b05bb
Received   =  0x1856b3445b36ddba62df50f70774111c1ade92b494771f5751c83677275c735d9b767360491fa4ad40da5220cdef05a45b8847f1fb9e6d9
f0400195172f2b05bb
```

Run result demo

2. 128bit subtraction

The second operation is subtraction. To perform this, we set the switch to the closed position (0'b1). The implementation method involves placing a multiplexer in front of the 128-bit adder. For subtraction, we invert all bits of B. Additionally, another multiplexer is positioned in front of the mp_adder. If it's a subtraction operation, the carry_in is fixed to 1. This combination ensures the generation of a 2's complement number.

```
A     =  0xb29badfadc986f408ca23618f6f5f14e042a27d94cd603629e4e0ad5ca51e059cfb41f0cdc7bcf9458536a3ab6ae85e566b6104cbee4dc08f24b
1e1c8decf46e
B     =  0xaecf635b0d5af185faf96e2cdceeb17f7b116f2031bf67064a3d46201712055d4d5bdaeb04a24fd95b207a448b949300441a0c8ac04560023f2c
4c94368dac77
A - B =  0x3cc4a9fcf3d7dba91a8c7ec1a073fce8918b8b91b169c5c5410c4b5b33fdafc82584421d7d97fbafd32eff62b19f2e5229c03c1fe9f7c06b31ed
188575f47f7
```

```
Result received correctly!
Expected   =  0x3cc4a9fcf3d7dba91a8c7ec1a073fce8918b8b91b169c5c5410c4b5b33fdafc82584421d7d97fbafd32eff62b19f2e5229c03c1fe9f7c06
b31ed188575f47f7
Received   =  0x3cc4a9fcf3d7dba91a8c7ec1a073fce8918b8b91b169c5c5410c4b5b33fdafc82584421d7d97fbafd32eff62b19f2e5229c03c1fe9f7c06
b31ed188575f47f7
```

Run result demo

Performance evaluation

The performance assessment relies on a post-synthesis timing report, ensuring that any physical or structural optimizations made during the implementation phase do not influence the evaluation. The mp_adder module was set as the top module for analysis, with a timing constraint set at 125 MHz.

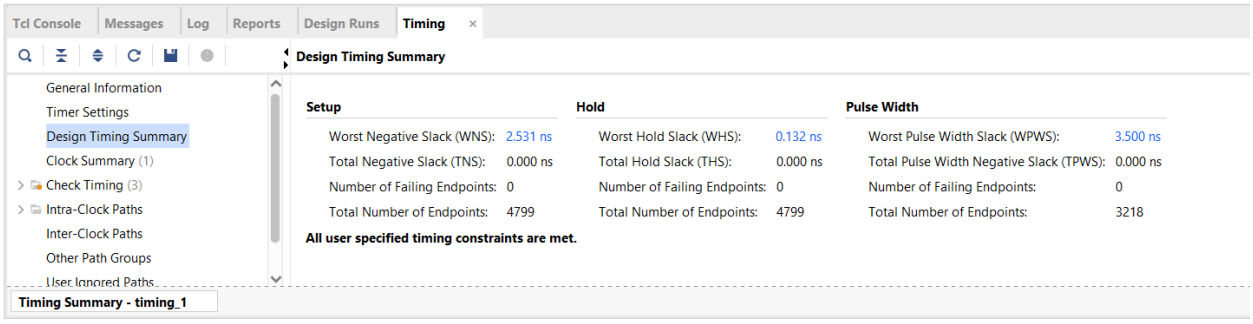


Figure 1: adder width 16 bits

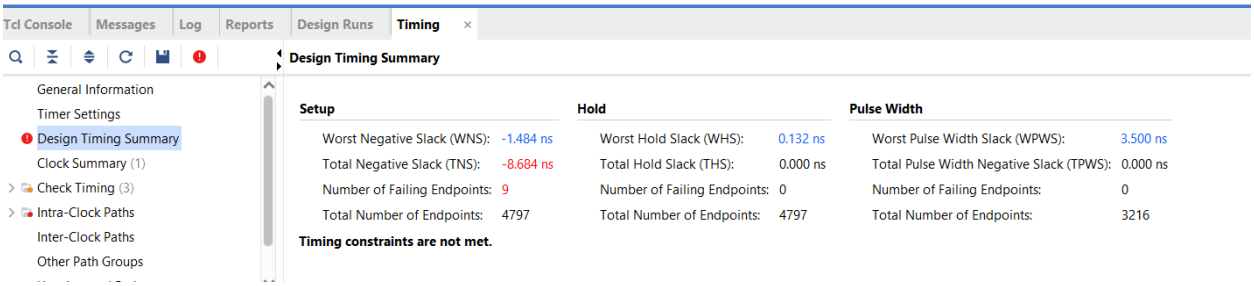
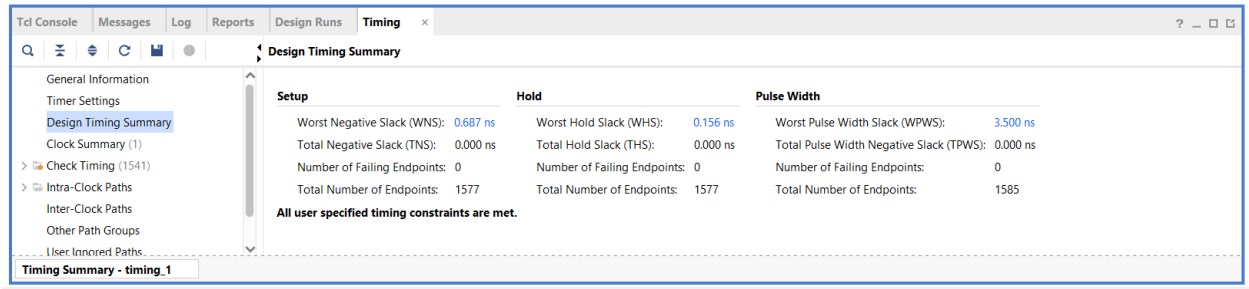
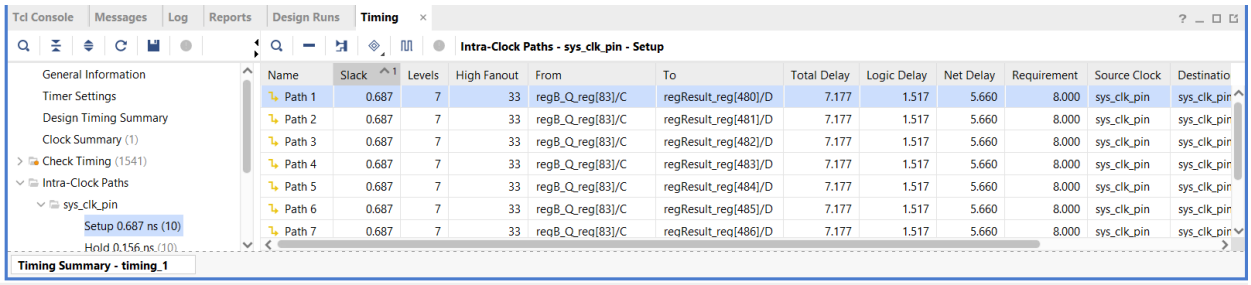


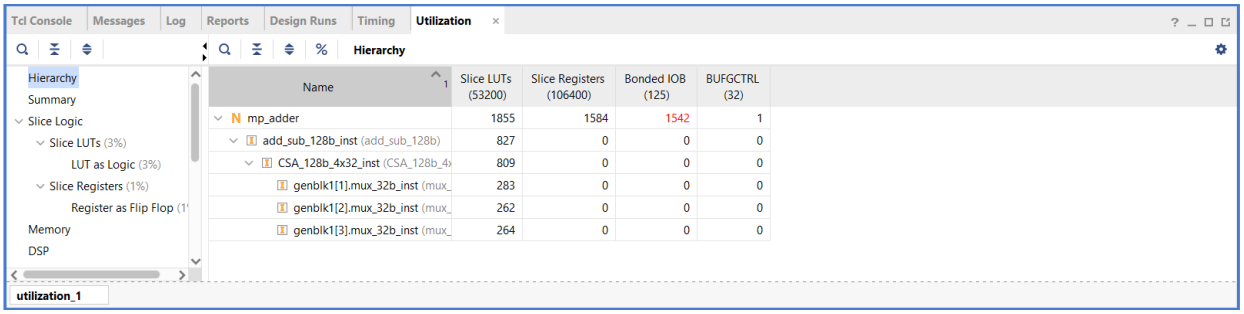
Figure 2: adder width = 32 bits BEFORE implementation

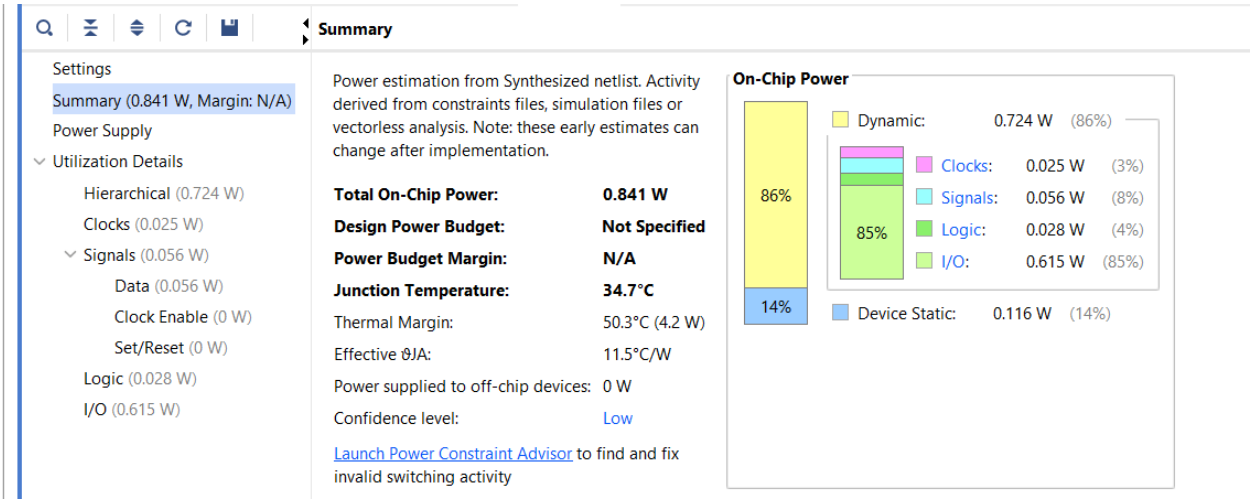
After implementation:

we use 8-bit non-iterative CLA → 32-bit CSA → 128-bit CSA.









Comparison with original performance metrics

The architecture of an adder plays a crucial role in its worst negative slack (WNS) and overall speed performance. In our analysis, we found that the Carry Lookahead Adder (CLA) emerged as a key contender to replace a full adder due to its inherent advantages. By integrating CLA into block modules within the Carry Select Adder (CSelA), it was anticipated that pre-fixed computing would result in a more favorable propagation delay performance compared to a ripple carry adder.

Our hypothesis was supported by the data, especially when the propagation logic within CLA was expanded. Generally, the expanded HYBRID adder demonstrated superior speed compared to the for-loop HYBRID adder, with the ripple carry adder being the slowest among them at the same adder width. Table presents a comparative overview of performance metrics such as slack, propagation delay, and area utilization The worse WNS compared to the 16-bit adder is due to the additional multiplexers and the increased adder width. Moreover, more LUTs are utilized because the lookahead logic is essentially implemented as a lookup table on the FPGA. Consequently, a significantly larger area is consumed as we are employing a greater number of logic gates.